Rudra Kathoke

A9-13

Loan Default Prediction using Support Vector Machines (SVM)

This notebook demonstrates a complete workflow to predict loan defaults based on financial and demographic data using SVM.

Project Steps:

1. **Data Generation & EDA**: Creating synthetic data mimicking financial/demographic indicators and analyzing distributions.
2. **Model Implementation**: Building a baseline SVM classifier.
3. **Evaluation**: Assessing the baseline using Precision, Recall, F1-score, Accuracy, and a Confusion Matrix.
4. **Hyperparameter Tuning & Kernel Comparison**: Testing different kernel types (`linear`, `rbf`, `poly`) and regularization strengths (C) to optimize performance.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.model_selection import GridSearchCV

np.random.seed(42)

n_samples = 1000

data = {
    'Age': np.random.randint(21, 65, size=n_samples),
    'Annual_Income': np.random.normal(55000, 15000, size=n_samples).round(2),
    'Credit_Score': np.random.randint(300, 850, size=n_samples),
    'Debt_to_Income_Ratio': np.random.uniform(0.1, 0.6, size=n_samples).round(2),
    'Employment_Duration_Years': np.random.randint(0, 20, size=n_samples)
}

df = pd.DataFrame(data)

risk_score = (0.5 * (850 - df['Credit_Score']) / 550) + (0.3 * df['Debt_to_Income_Ratio'] / 0.6) - (0.2 * df['Annual_Income'] / 
noise = np.random.normal(0, 0.1, size=n_samples)
final_score = risk_score + noise

df['Default'] = (final_score > final_score.median()).astype(int)

print("Dataset preview:")
display(df.head())
print(f"\nClass distribution:\n{df['Default'].value_counts(normalize=True)}")
```

```
Dataset preview:
```

	Age	Annual_Income	Credit_Score	Debt_to_Income_Ratio	Employment_Duration_Years	Default
0	59	41727.76	683	0.41	19	0
1	49	59304.12	744	0.39	19	0
2	35	57477.92	724	0.52	5	1
3	63	63534.05	460	0.55	3	1
4	28	41877.90	477	0.11	1	1

```
Class distribution:
Default
0    0.5
1    0.5
Name: proportion, dtype: float64
```

Exploratory Data Analysis (EDA)

We will visualize distributions of numerical features grouped by the target class to understand feature separation.

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

features = ['Age', 'Annual_Income', 'Credit_Score', 'Debt_to_Income_Ratio', 'Employment_Duration_Years']

for i, col in enumerate(features):
    sns.boxplot(x='Default', y=col, data=df, ax=axes[i], palette='Set2')
    axes[i].set_title(f'{col} vs Default')

axes[-1].set_visible(False)
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_3049/2046969681.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(x='Default', y=col, data=df, ax=axes[i], palette='Set2')
/tmp/ipykernel_3049/2046969681.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(x='Default', y=col, data=df, ax=axes[i], palette='Set2')
/tmp/ipykernel_3049/2046969681.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

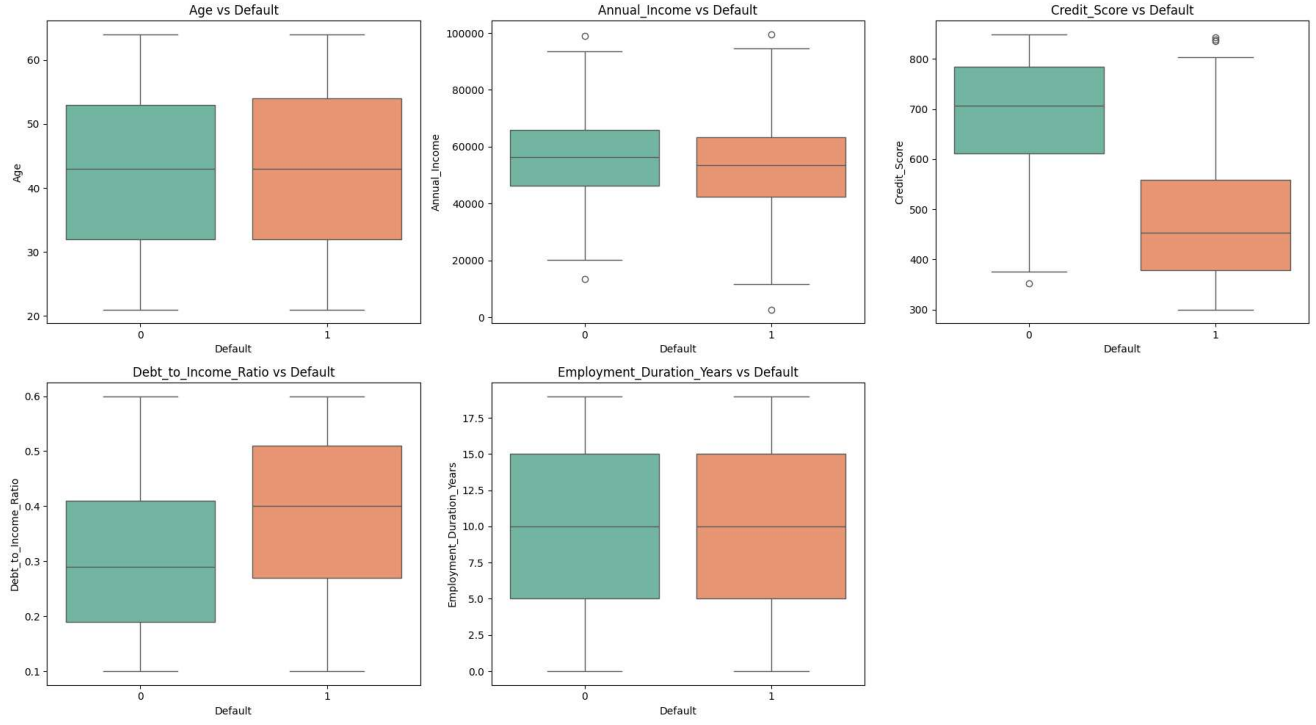
  sns.boxplot(x='Default', y=col, data=df, ax=axes[i], palette='Set2')
/tmp/ipykernel_3049/2046969681.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(x='Default', y=col, data=df, ax=axes[i], palette='Set2')
/tmp/ipykernel_3049/2046969681.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(x='Default', y=col, data=df, ax=axes[i], palette='Set2')
```



Data Preprocessing & Model Training

SVM is highly sensitive to the scale of the features, so we apply standard scaling before model training.

```
X = df[features]
y = df['Default']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

base_svm = SVC(kernel='rbf', C=1.0, random_state=42)
base_svm.fit(X_train_scaled, y_train)
y_pred = base_svm.predict(X_test_scaled)

print("Baseline Accuracy (80/20 Split):", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

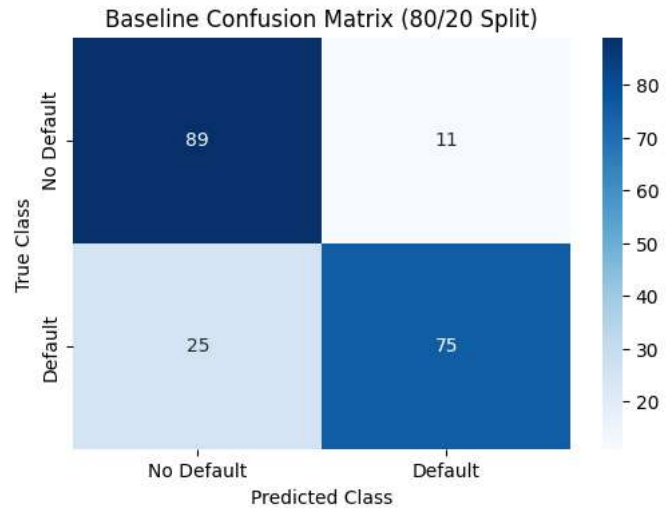
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Default', 'Default'], yticklabels=['No Default', 'Default']
plt.title('Baseline Confusion Matrix (80/20 Split)')
plt.ylabel('True Class')
plt.xlabel('Predicted Class')
plt.show()
```

```
Baseline Accuracy (80/20 Split): 0.82

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.89      0.83       100
           1       0.87      0.75      0.81       100

    accuracy                           0.82       200
   macro avg       0.83      0.82      0.82       200
weighted avg       0.83      0.82      0.82       200
```



Hyperparameter Tuning and Kernel Comparison

We will evaluate SVM configurations across multiple kernels (`linear`, `rbf`, `poly`) and dynamic ranges of the regularization parameter `C` using GridSearchCV.

```
param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf', 'poly'],
    'gamma': ['scale', 'auto']
}

grid = GridSearchCV(SVC(random_state=42), param_grid, cv=5, scoring='f1', n_jobs=-1)
grid.fit(X_train_scaled, y_train)

results_df = pd.DataFrame(grid.cv_results_)
```

```
print(f"Best parameters found: {grid.best_params_}")
print(f"Best F1 validation score: {grid.best_score_:.4f}")

pivot_results = results_df.pivot_table(index='param_C', columns='param_kernel', values='mean_test_score')
print("\nMean Test Scores (F1) across Kernels and C values:")
display(pivot_results)

best_model = grid.best_estimator_
y_pred_tuned = best_model.predict(X_test_scaled)

print("\nTuned Model Evaluation on Test Set:")
print("Tuned Accuracy:", accuracy_score(y_test, y_pred_tuned))
print("\nTuned Classification Report:\n", classification_report(y_test, y_pred_tuned))
```

```
Best parameters found: {'C': 0.1, 'gamma': 'auto', 'kernel': 'poly'}
Best F1 validation score: 0.8475

Mean Test Scores (F1) across Kernels and C values:
```

param_kernel	linear	poly	rbf
param_C			
0.1	0.841728	0.845536	0.839390
1.0	0.840175	0.829306	0.837785
10.0	0.841231	0.826380	0.825594
100.0	0.842657	0.826014	0.778792

```
Tuned Model Evaluation on Test Set:
Tuned Accuracy: 0.81

Tuned Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.85      0.82       100
           1       0.84      0.77      0.80       100

    accuracy                           0.81       200
   macro avg       0.81      0.81      0.81       200
weighted avg       0.81      0.81      0.81       200
```

Start coding or generate with AI.